Project3

Process and Process Management

Project3 Deadline: (Optional) 4/23/2026, 2:00 pm

Assignments

0. Merge Previous Project (project 1 & 2)

- How to verify:

  Step 1. *make test*

```
ch5_mergetest
ch5_ppid
ch5_setprio
ch5_spawn0
ch5_spawn1
ch5_usertest
ch5b_exec_simple
ch5b_exit
ch5b_forktest0
ch5b_forktest1
ch5b_forktest2
ch5b_getpid
ch5b_usertest
ch5t_stride0
ch5t_stride1
ch5t_stride2
ch5t_stride3
ch5t_stride4
ch5t_stride5
ch5t_usertest
usershell
C user shell
>>
```

  Step 2. *ch5_mergetest*

```
Usertests: Running ch4_mmap2
[ERROR 66]13 in application, bad addr = 0x0000000010000000, bad instruction = 0x0000000000001020, core dumped.
Usertests: Test ch4_mmap2 in Process 66 exited with code -2
ch5 Mergetests passed!
Shell: Process 2 exited with code 0
>>
```

0. Merge Previous Project (project 1 & 2)

os > C vm.c

```
166  {
183  }
184
185  // Recursively free page-table pages.
186  // All leaf mappings must already have been removed.
187  void freewalk(pagetable_t pagetable)
188  {
189      // there are 2^9 = 512 PTEs in a page table.
190      for (int i = 0; i < 512; i++) {
191          pte_t pte = pagetable[i];
192          if ((pte & PTE_V) && (pte & (PTE_R | PTE_W | PTE_X)) == 0) {
193              // this PTE points to a lower-level page table.
194              uint64 child = PTE2PA(pte);
195              freewalk((pagetable_t)child);
196              pagetable[i] = 0;
197          } else if (pte & PTE_V) {
198              //panic("freewalk: leaf");
199          }
200      }
201      kfree((void *)pagetable);
202  }
```

REMINDER

① **Do NOT copy the whole file from previous project,** otherwise the code updated will be overwritten

② **Comment this line before test**

1. Process creation

Implement a system call called "spawn" to create a new process.

```
int spawn(char *name)
```

- **syscall ID:** 400
  - **Functionality:** Equivalent to fork + exec, creates a new child process and executes the target program.
  - **Description:** Returns the process ID of the child if successful, otherwise returns -1.

1. Process creation

```
int spawn(char *name)
```

- **Possible errors:**
  - Invalid filename
  - Resource errors such as a full process pool or insufficient memory.

2. Stride scheduling algorithm

implement a priority-based scheduling algorithm

- **Description:**
  - For each process, set a current stride, indicating the "length" the process has run so far.
  - Set its corresponding pass value (which is only related to the process's priority), indicating the accumulated value that the stride needs to increase after scheduling the corresponding process.

2. Stride scheduling algorithm

- **Description:**
  - Whenever scheduling is required, select the process with the smallest stride from the currently runnable processes for scheduling.
  - For the scheduled process P, add its corresponding stride to its pass value.
  - After one time slice, return to the previous step and reschedule the process with the smallest current stride.

2. Stride scheduling algorithm

- It can be proven that if we set P.pass = BigStride / P.priority, then the time allocated to each process under this scheduling scheme will be proportional to its priority.

  - P.pass is the pass value of the process
  - P.priority is the priority of the process (greater than 1)
  - BigStride is a predefined large constant

2. Stride scheduling algorithm

- Other experimental details:
  - Setting process priorities is prohibited in stride scheduling, as it leads to errors.
  - Process initial stride should be set to 0.
  - Process initial priority should be set to 16

2. Stride scheduling algorithm

Implement a system call called ”sys_set_priority”

```
int setpriority(long long prio)
```

- **syscall ID:** 140
  - **Functionality:** Set process priority.
  - **Description:** Returns the process ID of the child if successful, otherwise returns -1.

2. Stride scheduling algorithm

Implement a system call called ”sys_set_priority”

```
int setpriority(long long prio)
```

- **syscall ID:** 140
  - **Explanation:** Sets the priority of the calling process.
    - If prio is within the range [2, isize_max], the operation is successful, and the function returns prio.
    - Otherwise, it returns -1.

- **Relevant test case**: ch5_setprio

Grading

Input:

1. make test
2. ch5t_usertest
3. ch5_usertest

```
>> ch5t_usertest
priority = 6, exitcode = 6925200
priority = 5, exitcode = 5402800
priority = 8, exitcode = 9238400
priority = 7, exitcode = 8238400
priority = 9, exitcode = 10643200
priority = 10, exitcode = 11718000
ch5t usertest passed!
Shell: Process 161 exited with code 0
```

```
Test sleep OK!
priority = 6, exitcode = 6920800
priority = 5, exitcode = 5253600
priority = 8, exitcode = 8913600
priority = 7, exitcode = 8070000
priority = 9, exitcode = 10228800
priority = 10, exitcode = 11797600
ch5t usertest passed!
ch5 Usertests passed!
Shell: Process 169 exited with code 0
```

Ideal outcome:

After all 6 processes exit,
the output count is proportional to their priorities.